

Weak Session ID

A **weak session ID** (or predictable session token) occurs when the server generates session identifiers with insufficient entropy—making them guessable or predictable. Good tokens should be long, random strings (e.g., 128+ bits of entropy). Weak ones might be sequential, timestamp-based, or use low-randomness sources.

This falls under OWASP's A07:2025 – Authentication Failures. Related but distinct: **Session Fixation** (attacker forces a known ID on the victim).

History & Evolution

- **Past:** Very common. Early apps used sequential IDs (e.g., 1, 2, 3) or weak algorithms (MD5 of timestamp + IP). Tools like WebScarab exposed predictability easily.
- **Present:** Much rarer in modern frameworks (they use cryptographically secure PRNGs). Still appears in custom code, legacy systems, or misconfigurations. Recent examples (2025–2026):
 - CVE-2025-24859 (Apache Roller): Allowed persistent sessions via weak handling.
 - CVE-2025-15387 (QNO VPN): Insufficient entropy in session IDs.
 - Other insufficient entropy issues in various products (e.g., thinbus-srp library).
- **Future Trends:** Declining further with token-based auth (JWTs, OAuth). But JWT misconfigurations (weak signing keys) create similar risks. AI-assisted entropy analysis may help defenders, while attackers use ML for prediction.

How It Works (Step-by-Step)

1. User logs in → server generates session ID (cookie/token).
2. Attacker collects multiple session IDs (e.g., by creating accounts).
3. Analyzes for patterns: sequential, encoded timestamps, low randomness.
4. Predicts/guesses valid IDs → hijacks active sessions.

Example Weak Token Patterns:

- Sequential: session=1001, 1002...
- Timestamp-based: session=1640995200-user1
- Low entropy: Short base64 with predictable parts.

Exploitation Procedure

1. **Gather Samples** – Register/log in multiple times, capture tokens (Burp Proxy).
2. **Analyze Entropy** – Use tools to test randomness.

3. **Exploit:**
 - If sequential: Guess nearby IDs.
 - If time-based: Generate during same window.
 - Brute-force short tokens.
4. **Hijack** – Set cookie to guessed token → access victim's session.

Differences from Similar Issues

Issue	Requires Stealing Token?	Needs Prediction?	Typical Vector	Primary Defense
Weak Session ID	No	Yes	Guessing/prediction	High entropy + regeneration
Session Hijacking (via XSS/CSRF/MITM)	Yes	No	Steal valid token	HttpOnly/Secure/SameSite cookies
Session Fixation	No	No	Force known ID	Regenerate ID on login
Broken Auth (Weak Passwords)	No	No	Credential stuffing	Strong passwords + MFA

Weak IDs enable hijacking without initial access.

Impacts & Real-World Effects

- Full account takeover.
- Data theft, privilege escalation.
- Lateral movement in breaches.
- Combined with other vulns for persistence.

In 2025 cases (e.g., Apache Roller), led to unauthorized persistent access.

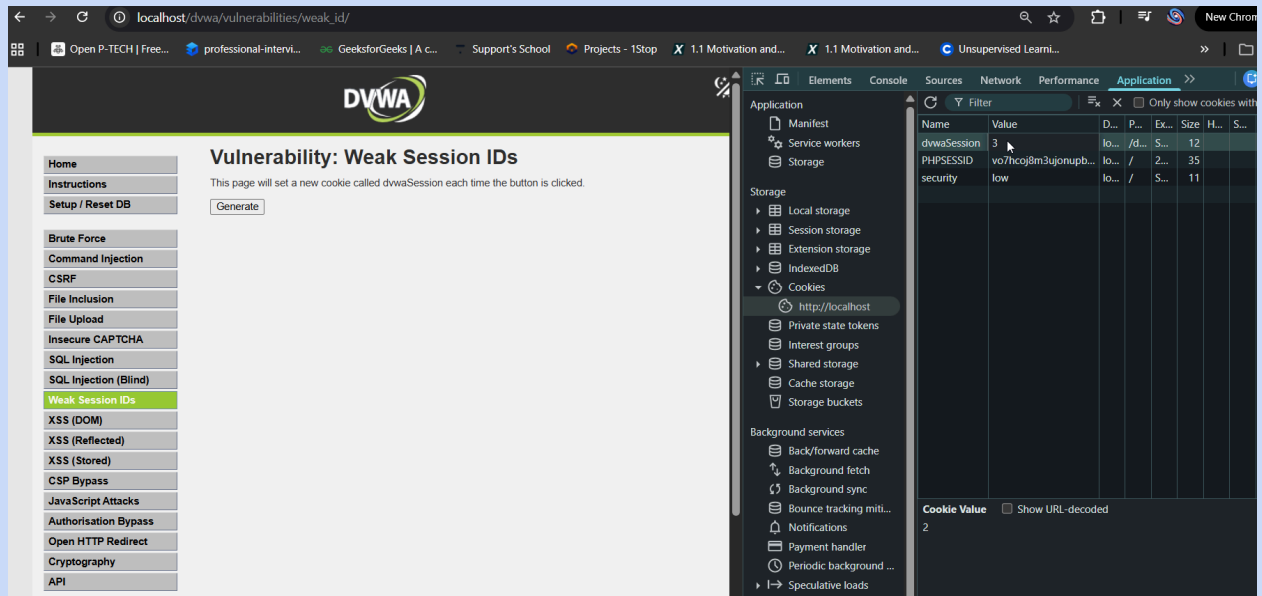
Example: DVWA lab

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface in a browser. The page title is "Vulnerability: Weak Session IDs". The description states: "This page will set a new cookie called dvwaSession each time the button is clicked." A "Generate" button is visible. The left sidebar contains a list of vulnerability categories, with "Weak Session IDs" highlighted. The right sidebar shows the Chrome DevTools "Application" tab, displaying the "Cookies" section for the current page. The cookies table is as follows:

Name	Value	Domain	Path	Expires	Size	HttpOnly	Secure	SameSite	Priority	Group
dvwaSession	1	localhost	/	Never	12					
PHPSESSID	vo7hcoj8m3ujonupb...	localhost	/	2...	35					
security	low	localhost	/	Never	11					

The screenshot shows the DVWA interface after clicking the "Generate" button. The "dvwaSession" cookie value has changed from "1" to "2". The "PHPSESSID" and "security" cookies remain unchanged. The "Application" tab in Chrome DevTools shows the updated cookies table:

Name	Value	Domain	Path	Expires	Size	HttpOnly	Secure	SameSite	Priority	Group
dvwaSession	2	localhost	/	Never	12					
PHPSESSID	vo7hcoj8m3ujonupb...	localhost	/	2...	35					
security	low	localhost	/	Never	11					



Summary:

Weak session IDs are less common in 2026 thanks to better frameworks, but they persist in legacy/custom code (as seen in 2025 CVEs). The core lesson: randomness is critical for security. Master entropy analysis with Burp Sequencer and always implement secure session handling.

Key takeaway: **Session tokens must be unguessable – treat them like passwords.**